

BAŞLIK: PYTHON İLE API TÜKETMEK: REQUESTS KÜTÜPHANESİ VE DAYANIKLI İSTEMCİ TASARIMI

TÜR: Uygulamalı/Teknik Detaylı Anlatım

BÖLÜM 1: REQUESTS KÜTÜPHANESİNE GİRİŞ

Python'da bir API'ye istek atmanın standart yolu, üçüncü parti **requests** kütüphanesidir (`pip install requests`). Python'ın yerleşik `urllib` modülü de aynı işi yapabilir ama `requests`, çok daha okunabilir ve "pythonic" bir arayüz sunduğu için (kendi belgelerinde "HTTP for Humans" olarak tanıtılır) endüstri standardı haline gelmiştir.

1.1. Temel İstek Atma

Her HTTP metoduna karşılık gelen bir fonksiyon vardır: `requests.get()`, `requests.post()`, `requests.put()`, `requests.patch()`, `requests.delete()`. Her biri bir `Response` nesnesi döner; bu nesne üzerinden durum kodunu (`.status_code`), gövdeyi JSON olarak (`.json()`) ya da ham metin olarak (`.text`) okuyabilirsiniz.

1.2. Sorgu Parametreleri, Başlıklar ve Gövde Gönderme

`requests`, URL'yi elle string birleştirerek (`"?sayfa=" + str(sayfa)` gibi) oluşturmak yerine, `params` parametresiyle bir sözlük (dict) vermenizi sağlar — kütüphane, doğru URL kodlamasını (özel karakterlerin kaçış/escape edilmesi dahil) sizin için otomatik yapar. Aynı şekilde `headers` parametresiyle başlıklar, `json` parametresiyle de POST/PUT gövdesi (otomatik olarak JSON'a çevrilir VE `Content-Type: application/json` başlığı otomatik eklenir) gönderilir.

BÖLÜM 2: NEDEN 'NAİF' BİR İSTEMCİ YETERLİ DEĞİLDİR?

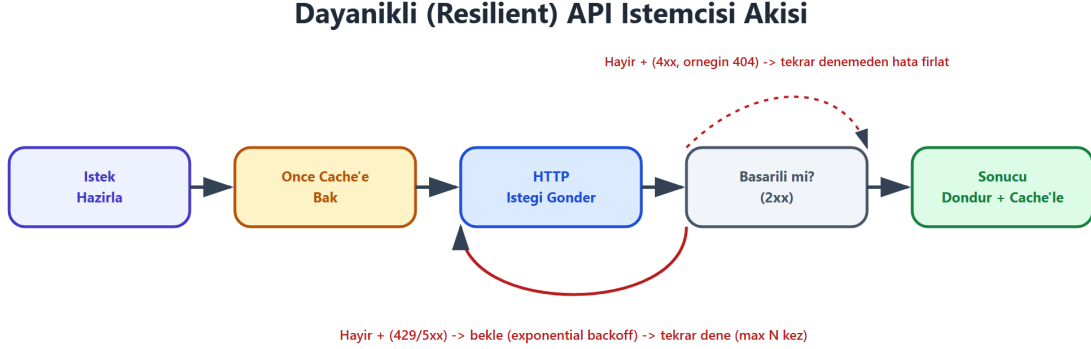
Bir eğitim ortamında `requests.get(url).json()` yazıp geçmek yeterli görünebilir. Ama PRODUCTION'da (gerçek, canlı bir sistemde) çalışan bir API istemcisi, ağın ve dış sistemlerin doğası gereği GÜVENİLMEZ olduğunu varsaymak zorundadır. Karşılaşılabilecek gerçek dünya sorunları:

- Geçici ağ hataları: Bağlantı zaman aşımına uğrayabilir, DNS çözümülemesi başarısız olabilir, paket kaybı yaşanabilir — bunların çoğu GEÇİCİDİR ve birkaç saniye sonra tekrar denenirse başarılı olur.
- Sunucu tarafı geçici hatalar (5xx): Sunucu o an aşırı yüklenmiş olabilir (503 Service Unavailable) — bu da genellikle geçicidir.
- Hız sınırı aşımı (429): Sunucu "şu an çok hızlı istek atıyorsun, yavaşla" diyor (bkz. 22.4, Bölüm 3) — bu bir HATA değil, bir SİNYALdir; doğru cevap beklemek ve tekrar denemektir.
- Kalıcı hatalar (4xx): 404 Not Found ya da 401 Unauthorized gibi hatalar TEKRAR DENENEREK düzelmez — isteği aynen tekrar göndermek sadece kaynak israfıdır ve sunucuyu gereksiz yere daha fazla yorar.
- Zaman aşımı (timeout) belirtilmemesi: `requests`'te varsayılan olarak HİÇBİR zaman aşımı yoktur! Sunucu yanıt vermeyi unutursa, isteğiniz SONSUZA kadar bekler ve programınız donar ("hang" olur). Her istekte MUTLAKA bir timeout belirtilmelidir.

İyi bir API istemcisi bu senaryoların HER BİRİNİ farklı şekilde ele almalıdır — bu, aşağıdaki "dayanıklı istemci" (resilient client) mimarisinin temelidir.

BÖLÜM 3: DAYANIKLI (RESILIENT) API İSTEMCİSİ MİMARİSİ

22.5.2 dosyasındaki uygulamalı projede, aşağıdaki akışı takip eden, gerçek dünya standartlarında bir API istemci sınıfı yazacağız:



Şekil 1 — Dayanıklı bir API istemcisinin karar akışı: önce önbellek, sonra istek, hataya göre farklı tepki (tekrar dene / hemen vazgeç).

3.1. Oturum (Session) Kullanımı

`requests.get()` her çağrıldığında YENİ bir TCP bağlantısı açar. Aynı sunucuya art arda çok sayıda istek atıyorsanız (örn. sayfalama ile 50 sayfa çekmek), bu çok verimsizdir. `requests.Session()` nesnesi, bağlantıları YENİDEN KULLANARAK (connection pooling / keep-alive) performansı ciddi şekilde artırır; ayrıca ortak başlıkları (örn. Authorization) session üzerinde bir kere ayarlayıp her istekte tekrar yazmaktan kurtulursunuz.

3.2. Üstel Geri Çekilme ile Tekrar Deneme (Exponential Backoff Retry)

Bir istek başarısız olduğunda HEMEN tekrar denemek kötü bir fikirdir — eğer sunucu zaten aşırı yüklenmişse, yüzlerce istemcinin AYNI ANDA tekrar denemesi durumu daha da kötüleştirir ("thundering herd" - kükreyen sürü - problemi). Üstel geri çekilme, her başarısız denemeden sonra bekleme süresini katlayarak artırır (örn. 1sn, 2sn, 4sn, 8sn...) ve genellikle küçük bir rastgelelik (jitter) eklenir ki tüm istemciler TAM OLARAK aynı anda tekrar denemesin. Python'da bunu elle yazabileceğimiz gibi, `urllib3`'ün `Retry` sınıfı ve `HTTPAdapter` ile `requests` seviyesinde OTOMATİK hale de getirebiliriz — 22.5.2'de her iki yaklaşımı da göreceğiz.

3.3. Hangi Hatalarda Tekrar Denenir?

Kritik kural: SADECE geçici olduğu düşünülen hatalarda tekrar denenir (429, 500, 502, 503, 504 ve ağ bağlantı hataları). KALICI hatalarda (400, 401, 404) tekrar denemek anlamsızdır — isteği olduğu gibi tekrar göndermek yine aynı hatayı üretir. İyi bir istemci bu ayrımı YAPMAK ZORUNDADIR.

3.4. Basit Önbellekleme (Caching)

Sık değişmeyen veriler için (örn. bir ürün kataloğu, bir kullanıcının profil bilgisi), her seferinde ağa gitmek yerine, sonucu bir süre yerel bellekte (in-memory) tutmak hem hızı artırır hem sunucu üzerindeki yükü azaltır hem de hız sınırına takılma riskini düşürür. 22.5.2'deki istemcimiz, basit bir zaman-aşım (TTL - time to live) önbellek katmanı içerecek.

3.5. Sayfalama Otomasyonu

Bir koleksiyonun TÖM sayfalarını çekmek istediğinizde (bkz. 22.3, Bölüm 4), her sayfayı elle istemek yerine, bir Python generator (yield kullanan bir fonksiyon) ile "bir sonraki sayfa var mı?" mantığını soyutlamak, kullanım kolaylığı açısından büyük fark yaratır — çağırın kod sadece for kayıt in tum_kayitlari_getir(): yazar, sayfalama detaylarıyla hiç uğraşmaz.

SONUÇ

requests kütüphanesinin temellerini ve production kalitesinde bir API istemcisinin sahip olması gereken 5 temel özelliği (session, akıllı tekrar deneme, hata ayrımı, önbellekleme, sayfalama otomasyonu) öğrendik. Sıradaki dosyada (22.5.2), bu teoriyi tamamen çalışan, test edilmiş bir Python sınıfına dönüştüreceğiz.